

HCL TP1 Java Interview

10 Years Experience — 160+ Questions, Solutions & Mock Interview

203 technical questions covering Core Java, Java 8+, Collections, Concurrency, Spring Boot, REST, Microservices, JPA/Hibernate, SQL, coding, production troubleshooting and system design.

Interview focus: At senior level, answer with the concept, internal working, real project example, trade-offs, failure handling and testing/monitoring approach.

How to Use This Guide

For each question, first answer aloud in 60–90 seconds. Then use the solution as a reference. For coding problems, write the solution yourself before reading the approach. For senior scenarios, structure answers as: Situation → Analysis → Design/Action → Trade-off → Result.

Priority Revision Order

1) Core Java & Collections → 2) Java 8/Streams → 3) Multithreading → 4) Spring Boot → 5) Microservices → 6) SQL/JPA → 7) Coding → 8) Production/System Design.

1. Core Java & OOP

Q: Explain OOP principles.

Answer / Solution: Encapsulation hides state behind methods; inheritance reuses behavior; polymorphism lets the same contract have different implementations; abstraction exposes essential behavior while hiding implementation.

Q: Abstraction vs encapsulation.

Answer / Solution: Abstraction focuses on what an object does; encapsulation focuses on protecting how its state is stored and changed.

Q: Overloading vs overriding.

Answer / Solution: Overloading uses the same method name with different parameters and is resolved at compile time. Overriding replaces inherited behavior with the same signature and is resolved at runtime.

Q: Can a static method be overridden?

Answer / Solution: No. Static methods belong to the class. A subclass can hide a static method, but this is method hiding, not runtime overriding.

Q: Can a private method be overridden?

Answer / Solution: No. Private methods are not inherited and therefore cannot be overridden.

Q: final vs finally vs finalize.

Answer / Solution: final restricts variables, methods, or classes; finally is a block normally executed after try/catch; finalize was a GC-related hook and is deprecated/should not be relied on.

Q: Why is String immutable?

Answer / Solution: Immutability improves security, string-pool reuse, thread safety, and stable hash codes. Once created, a String's value cannot change.

Q: String vs StringBuilder vs StringBuffer.

Answer / Solution: String is immutable. StringBuilder is mutable and preferred for single-threaded concatenation. StringBuffer is mutable and synchronized, so generally slower.

Q: == vs equals().

Answer / Solution: For objects, == compares references; equals() compares logical equality when the class overrides it. For primitives, == compares values.

Q: Explain equals() and hashCode().

Answer / Solution: If two objects are equal according to equals(), they must have the same hashCode. Unequal objects may still share a hash code.

Q: How does HashMap work internally?

Answer / Solution: It hashes the key, maps the hash to a bucket, and stores entries in that bucket. Collisions are handled by linked nodes and, when appropriate, tree bins in modern Java.

Q: What happens during a HashMap collision?

Answer / Solution: Multiple keys can land in the same bucket. Java compares hashes and then keys using equals(); heavily populated buckets can be treeified subject to implementation thresholds.

Q: What if a HashMap key is mutated after insertion?

Answer / Solution: Lookup can fail because the key's hash/equality may no longer point to the original bucket. Keys should be immutable with respect to fields used by equals/hashCode.

Q: HashMap vs ConcurrentHashMap.

Answer / Solution: HashMap is not thread-safe. ConcurrentHashMap supports concurrent access with thread-safe operations and does not permit null keys or null values.

Q: HashMap vs Hashtable.

Answer / Solution: Hashtable is legacy synchronized Map and disallows nulls. HashMap is modern, generally faster, not synchronized, and permits one null key and multiple null values.

Q: ArrayList vs LinkedList.

Answer / Solution: ArrayList provides fast indexed access and usually better cache locality. LinkedList has node-based storage and is useful only for specific insertion/removal patterns; it is often slower in practice.

Q: HashSet vs TreeSet.

Answer / Solution: HashSet provides hash-based uniqueness with no sorted order guarantee. TreeSet maintains sorted order using a tree and requires comparable elements or a comparator.

Q: Comparable vs Comparator.

Answer / Solution: Comparable defines a class's natural ordering through compareTo(). Comparator defines external/custom ordering and supports multiple sorting strategies.

Q: Can HashMap contain null?

Answer / Solution: Yes. HashMap permits one null key and multiple null values. ConcurrentHashMap does not permit null keys or values.

Q: How do you create an immutable class?

Answer / Solution: Make the class final, fields private and final, initialize them through the constructor, provide no setters, and defensively copy mutable input/output objects.

Q: Checked vs unchecked exceptions.

Answer / Solution: Checked exceptions are verified by the compiler and must be caught or declared. RuntimeException subclasses are unchecked and generally represent programming or validation failures.

Q: throw vs throws.

Answer / Solution: throw actually throws an exception object. throws declares that a method may propagate specified exception types.

Q: What is try-with-resources?

Answer / Solution: It automatically closes AutoCloseable resources, even when an exception occurs. It is preferred for files, streams, JDBC resources, and similar resources.

Q: What is serialization?

Answer / Solution: Serialization converts an object state into a byte stream. Deserialization reconstructs it. Serializable classes should consider version compatibility and security implications.

Q: What is transient?

Answer / Solution: A transient field is excluded from default Java serialization.

Q: What is a record?

Answer / Solution: A record is a concise Java type for immutable data carriers, automatically providing accessors, equals, hashCode, and toString based on components.

2. Java 8+, Streams & Functional Programming

Q: List major Java 8 features.

Answer / Solution: Lambdas, functional interfaces, Stream API, Optional, default/static interface methods, method references, and the java.time API.

Q: What is a functional interface?

Answer / Solution: An interface with exactly one abstract method. It can be implemented with a lambda. Examples include Predicate, Function, Consumer, and Supplier.

Q: Predicate vs Function vs Consumer vs Supplier.

Answer / Solution: Predicate<T> returns boolean; Function<T,R> transforms T to R; Consumer<T> consumes a value without returning one; Supplier<T> supplies a value without input.

Q: map vs flatMap.

Answer / Solution: map transforms each element into one result. flatMap transforms each element into a stream and flattens the resulting streams into one stream.

Q: Intermediate vs terminal stream operations.

Answer / Solution: Intermediate operations such as filter/map are lazy and return a Stream. Terminal operations such as collect/count/forEach trigger execution.

Q: Stream vs Collection.

Answer / Solution: A Collection stores data; a Stream describes a computation over data. Streams are generally single-use and support lazy pipelines.

Q: Sequential vs parallel stream.

Answer / Solution: Sequential streams process in one thread. Parallel streams use the common ForkJoinPool and can help CPU-bound workloads with suitable data size and operations, but can hurt performance for small or blocking workloads.

Q: What is Optional?

Answer / Solution: Optional represents a value that may be absent and can make return-value absence explicit. It is most useful at API boundaries; avoid indiscriminate use for fields, parameters, and persistence models.

Q: findFirst vs findAny.

Answer / Solution: findFirst respects encounter order where applicable; findAny may return any matching element and can be more flexible for parallel processing.

Q: How does groupingBy work?

Answer / Solution: Collectors.groupingBy classifies elements by a key and accumulates values into a Map. A downstream collector can compute counts, sums, or nested groups.

Q: How do you remove duplicates with streams?

Answer / Solution: Use distinct() when equality semantics are correct, or collect into a Set when ordering requirements permit.

Q: How do you sort employees by salary?

Answer / Solution: Use `stream.sorted(Comparator.comparing(Employee::getSalary))` and collect the result.

Q: How do you find the highest salary?

Answer / Solution: Use `map(Employee::getSalary).max(Comparator.naturalOrder())` or `max(Comparator.comparing(Employee::getSalary))` on employees.

Q: How do you find the second-highest distinct salary?

Answer / Solution: Map to salary, `distinct()`, sort descending, `skip(1)`, then `findFirst()`.

Q: How do you group employees by department?

Answer / Solution: Use `Collectors.groupingBy(Employee::getDepartment)`.

Q: How do you convert `List<Employee>` to Map?

Answer / Solution: Use `Collectors.toMap(Employee::getId, Function.identity())` and provide a merge function if IDs may duplicate.

Q: What is lazy evaluation in streams?

Answer / Solution: Intermediate operations do not execute immediately. They execute when a terminal operation consumes the pipeline.

Q: When should you avoid parallel streams?

Answer / Solution: Avoid them for small collections, order-sensitive logic, shared mutable state, blocking I/O, or when the common pool is already under pressure.

Q: What is a method reference?

Answer / Solution: A compact lambda syntax such as `String::trim` or `Employee::getName` when an existing method matches the functional interface signature.

Q: What is effectively final?

Answer / Solution: A local variable used by a lambda or anonymous class must be final or not reassigned after initialization.

3. Multithreading & Concurrency

Q: Thread lifecycle.

Answer / Solution: A Java thread moves through NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED states depending on execution and synchronization.

Q: Thread vs Runnable vs Callable.

Answer / Solution: Thread represents execution, Runnable supplies work without a return value, and Callable supplies work that can return a value and throw checked exceptions.

Q: What is ExecutorService?

Answer / Solution: It manages worker threads and task submission, allowing thread reuse and controlled concurrency instead of creating a thread for every task.

Q: execute vs submit.

Answer / Solution: `execute` accepts Runnable and does not return a Future. `submit` accepts Runnable/Callable and returns a Future that can represent completion, result, or failure.

Q: What is Future?

Answer / Solution: Future represents an asynchronous result and supports checking completion, cancellation, and blocking retrieval.

Q: What is CompletableFuture?

Answer / Solution: It provides composable asynchronous stages such as thenApply, thenCompose, allOf, exceptionally, and handle.

Q: synchronized vs Lock.

Answer / Solution: synchronized is simple monitor-based locking. Lock APIs provide features such as tryLock, interruptible acquisition, and multiple conditions.

Q: What is volatile?

Answer / Solution: volatile guarantees visibility of writes across threads and establishes happens-before for accesses to that variable, but it does not make compound operations such as i++ atomic.

Q: What is a race condition?

Answer / Solution: The result depends on unpredictable interleaving of concurrent operations on shared state. Prevent it with immutability, synchronization, atomic classes, or proper concurrency design.

Q: What is deadlock?

Answer / Solution: Two or more threads wait forever for locks held by one another. Prevent it with consistent lock ordering, timeouts, reduced lock scope, and careful design.

Q: What is starvation?

Answer / Solution: A thread cannot obtain required CPU time or a lock because other threads continually get preference.

Q: CountdownLatch vs CyclicBarrier.

Answer / Solution: CountdownLatch waits until a count reaches zero and is generally one-use. CyclicBarrier lets a fixed set of threads wait for one another and can be reused.

Q: What is AtomicInteger?

Answer / Solution: A lock-free atomic variable supporting operations such as incrementAndGet and compareAndSet.

Q: What is ForkJoinPool?

Answer / Solution: A work-stealing pool designed for tasks that can be recursively split into smaller subtasks.

Q: How do you troubleshoot a production thread issue?

Answer / Solution: Capture thread dumps, identify BLOCKED/WAITING/RUNNABLE threads, inspect lock ownership and CPU, correlate with logs/metrics, then isolate contention, deadlock, or slow external calls.

Q: How do you analyze a thread dump?

Answer / Solution: Look for repeated stack traces, deadlock indicators, blocked threads, long-running RUNNABLE threads, lock owners, and application-specific call stacks.

Q: What is thread-safe design?

Answer / Solution: Prefer immutable state, stateless services, local variables, thread-safe collections, atomic operations, and narrowly scoped synchronization.

Q: What is a daemon thread?

Answer / Solution: A background thread that does not prevent JVM shutdown once all non-daemon threads finish.

4. Spring & Spring Boot

Q: Spring vs Spring Boot.

Answer / Solution: Spring provides the dependency-injection and application framework ecosystem. Spring Boot simplifies setup with auto-configuration, starters, embedded servers, and production-ready features.

Q: What does @SpringBootApplication do?

Answer / Solution: It combines configuration, component scanning, and auto-configuration annotations.

Q: How does Spring Boot auto-configuration work?

Answer / Solution: Boot examines the classpath, existing beans, properties, and conditional annotations to configure sensible defaults that can be overridden by application configuration.

Q: @Component vs @Service vs @Repository.

Answer / Solution: All register Spring beans. @Service communicates service-layer intent; @Repository identifies persistence components and enables persistence exception translation; @Component is generic.

Q: @Bean vs @Component.

Answer / Solution: @Component is detected by component scanning on a class. @Bean explicitly declares a bean from a configuration method.

Q: @Autowired vs constructor injection.

Answer / Solution: Constructor injection makes dependencies explicit, supports immutability, and is easier to test. It is generally preferred.

Q: @Qualifier vs @Primary.

Answer / Solution: @Primary provides a default candidate when multiple beans exist. @Qualifier selects a specific bean by qualifier.

Q: Spring bean lifecycle.

Answer / Solution: Instantiate, inject dependencies, invoke aware callbacks/post-processors, initialize, use, then destroy through configured lifecycle callbacks.

Q: Singleton vs Prototype scope.

Answer / Solution: Singleton creates one bean instance per Spring container. Prototype creates a new instance when requested; Spring does not manage the full destruction lifecycle of prototype beans.

Q: Explain @Transactional.

Answer / Solution: It defines a transactional boundary, usually implemented with a Spring proxy. The transaction commits on successful completion and rolls back according to configured rollback rules.

Q: What happens when an exception occurs in @Transactional?

Answer / Solution: By default, unchecked exceptions and Errors trigger rollback; checked exceptions generally do not unless configured with rollbackFor.

Q: Transaction propagation.

Answer / Solution: REQUIRED joins or creates a transaction; REQUIRES_NEW suspends the current one and creates a new transaction; SUPPORTS uses one if present; others define different participation behavior.

Q: Transaction isolation levels.

Answer / Solution: READ_UNCOMMITTED, READ_COMMITTED, REPEATABLE_READ, and SERIALIZABLE provide increasing isolation with different concurrency/locking trade-offs. Exact behavior depends on the database.

Q: @ControllerAdvice.

Answer / Solution: Provides centralized exception handling and cross-controller advice, commonly with @ExceptionHandler methods.

Q: How do you validate REST requests?

Answer / Solution: Use Bean Validation annotations such as @NotNull/@Size/@Valid and return structured validation errors from centralized exception handling.

Q: @Value vs @ConfigurationProperties.

Answer / Solution: @Value is convenient for individual properties. @ConfigurationProperties is better for binding and validating grouped configuration.

Q: How do Spring Profiles work?

Answer / Solution: Profiles activate environment-specific beans/properties such as dev, test, and prod.

Q: What is Spring Boot Actuator?

Answer / Solution: It exposes operational endpoints such as health, metrics, and info, subject to configuration and security.

Q: How do you implement pagination?

Answer / Solution: Use Pageable/Page with Spring Data or explicit LIMIT/OFFSET/keyset queries. For large datasets, keyset pagination can be more efficient.

Q: How do you implement global error handling?

Answer / Solution: Use @RestControllerAdvice with @ExceptionHandler methods, a consistent error DTO, HTTP status mapping, correlation ID, and safe client-facing messages.

5. REST APIs & Microservices

Q: Monolith vs microservices.

Answer / Solution: A monolith deploys a cohesive application as one unit. Microservices split business capabilities into independently deployable services, increasing autonomy but adding distributed-system complexity.

Q: How would you design microservices?

Answer / Solution: Split by business capability, define clear APIs and ownership, isolate data where appropriate, add observability/security, automate deployment, and design for failure.

Q: REST vs Feign client.

Answer / Solution: REST is an architectural style for HTTP APIs. Feign is a declarative Java HTTP client that simplifies calling REST services.

Q: RestTemplate vs WebClient vs Feign.

Answer / Solution: RestTemplate is a synchronous client and is in maintenance mode in modern Spring. WebClient supports reactive/non-blocking calls. Feign offers declarative synchronous clients, with ecosystem-specific alternatives for reactive needs.

Q: What is API Gateway?

Answer / Solution: A gateway provides a single entry point for routing, authentication, rate limiting, aggregation, and cross-cutting concerns.

Q: What is service discovery?

Answer / Solution: A mechanism by which services locate healthy instances dynamically, avoiding hard-coded host lists.

Q: What is Circuit Breaker?

Answer / Solution: It stops repeatedly calling a failing dependency after a threshold, allowing recovery and preventing cascading failure. It typically has closed, open, and half-open states.

Q: Retry vs Circuit Breaker.

Answer / Solution: Retry repeats transiently failed calls; circuit breaker prevents repeated calls when failure is persistent. Combining them requires bounded retries, backoff, and careful timeout settings.

Q: What is Saga pattern?

Answer / Solution: A distributed transaction is broken into local transactions coordinated by orchestration or choreography, with compensating actions for failures.

Q: How do you handle distributed transactions?

Answer / Solution: Prefer local transactions and asynchronous events; use Saga/outbox patterns when business consistency spans services rather than relying on a distributed 2PC transaction.

Q: How do you secure service-to-service calls?

Answer / Solution: Use TLS, OAuth2/client credentials or signed tokens, service identity, authorization scopes, secret rotation, and network controls.

Q: JWT authentication flow.

Answer / Solution: A client authenticates, receives a signed JWT, sends it with requests, and the resource server validates signature, issuer, audience, expiry, and required authorities.

Q: OAuth2 vs JWT.

Answer / Solution: OAuth2 is an authorization framework. JWT is a token format. OAuth2 access tokens can be JWTs, but OAuth2 does not require JWT.

Q: How do you version APIs?

Answer / Solution: Use backward-compatible evolution where possible; otherwise use URI, header, or media-type versioning with a documented deprecation policy.

Q: How do you maintain backward compatibility?

Answer / Solution: Avoid breaking field removals/renames, tolerate unknown fields, version incompatible contracts, and use contract tests.

Q: How do you trace requests across services?

Answer / Solution: Use correlation/trace IDs and distributed tracing such as OpenTelemetry, with logs and metrics linked to the same trace context.

Q: How do you prevent duplicate processing?

Answer / Solution: Use idempotency keys, unique constraints, deduplication tables, transactional outbox/inbox patterns, and carefully designed consumers.

Q: What is idempotency?

Answer / Solution: Repeating the same request produces the same intended business result. PUT is designed to be idempotent; POST can be made idempotent with an idempotency key.

Q: What is rate limiting?

Answer / Solution: It controls request volume per client/API to protect resources. Algorithms include token bucket and leaky bucket.

Q: How do you handle service timeouts?

Answer / Solution: Set bounded connection/read timeouts, use cancellation, fallback only when semantically safe, monitor latency, and avoid unbounded retries.

Q: What is eventual consistency?

Answer / Solution: Different services may temporarily observe different states, but the system converges after successful propagation and processing.

Q: What is the Outbox pattern?

Answer / Solution: Write the business change and an outbox event in the same local transaction, then publish the event asynchronously from the outbox.

6. JPA, Hibernate & Database

Q: JPA vs Hibernate.

Answer / Solution: JPA is a standard persistence specification. Hibernate is a widely used JPA implementation with additional features.

Q: save vs saveAndFlush.

Answer / Solution: save schedules/persists an entity according to the persistence context. saveAndFlush forces synchronization with the database immediately, though the surrounding transaction may still not be committed.

Q: Lazy vs Eager loading.

Answer / Solution: Lazy loads associations when accessed; eager loads them earlier. Lazy is generally safer for large relationships but requires proper transaction/query design.

Q: What is N+1 problem?

Answer / Solution: One query loads parent records followed by one query per parent for a relationship, causing many database calls.

Q: How do you solve N+1?

Answer / Solution: Use fetch joins/entity graphs, batch fetching, DTO projections, or carefully designed queries depending on the use case.

Q: @OneToMany vs @ManyToOne.

Answer / Solution: One department may have many employees, while each employee belongs to one department. In relational mappings, the many side normally owns the foreign key.

Q: Cascade types.

Answer / Solution: Cascade propagates persistence operations such as persist, merge, remove, refresh, and detach from parent to associated entities. Use REMOVE carefully.

Q: orphanRemoval.

Answer / Solution: Removing a child from the parent's relationship can cause the child row to be deleted, depending on mapping and provider behavior.

Q: First-level vs second-level cache.

Answer / Solution: First-level cache is tied to a persistence context/entity manager. Second-level cache is shared across persistence contexts and requires explicit provider configuration.

Q: Optimistic vs pessimistic locking.

Answer / Solution: Optimistic locking detects conflicts using a version field. Pessimistic locking acquires database locks to prevent conflicting operations.

Q: JPQL vs native query.

Answer / Solution: JPQL queries entities and their relationships in a database-independent style. Native SQL queries database-specific tables/features.

Q: Projection.

Answer / Solution: A projection retrieves only required fields instead of full entities, reducing data transfer and sometimes improving performance.

Q: LazyInitializationException.

Answer / Solution: It occurs when a lazy association is accessed outside an active persistence context. Fix it with proper transaction boundaries, fetch joins, entity graphs, or DTO queries—not by blindly making everything eager.

Q: Index basics.

Answer / Solution: Indexes accelerate reads for suitable predicates/orderings but consume storage and add write/update overhead.

Q: How do you troubleshoot slow SQL?

Answer / Solution: Inspect execution plan, indexes, row estimates, joins, predicates, statistics, locks, I/O, and query frequency; then benchmark the change.

Q: Normalization.

Answer / Solution: Normalization reduces redundancy and update anomalies. Common normal forms include 1NF, 2NF, and 3NF; denormalization can be used intentionally for performance.

Q: DELETE vs TRUNCATE vs DROP.

Answer / Solution: DELETE removes rows and can be filtered; TRUNCATE removes all rows using database-specific semantics; DROP removes the table/object itself. Transaction and logging behavior varies by database.

Q: Oracle to PostgreSQL migration.

Answer / Solution: Inventory schemas and SQL, map data types/functions/sequences, convert PL/SQL, test indexes and transactions, migrate data, validate counts/checksums, run performance tests, and plan cutover/rollback.

Q: What causes numeric input syntax errors in PostgreSQL?

Answer / Solution: A non-numeric string is being cast to a numeric type. Validate/clean input and use appropriate NULL/default handling rather than unsafe casts.

Q: Timestamp with/without time zone.

Answer / Solution: timestamp without time zone stores a date-time without timezone semantics; timestamptz represents an instant and is displayed in the session timezone. Choose based on business meaning.

7. SQL Interview Problems

Q: Find the second-highest salary.

Answer / Solution: SELECT MAX(salary) FROM employee WHERE salary < (SELECT MAX(salary) FROM employee); For distinct salary semantics, use DENSE_RANK or a DISTINCT subquery.

Q: Find the Nth-highest salary.

Answer / Solution: Use DENSE_RANK() OVER (ORDER BY salary DESC) and filter rank = N, or use a database-specific OFFSET/FETCH approach.

Q: Highest salary in each department.

Answer / Solution: Use ROW_NUMBER/DENSE_RANK partitioned by department, or GROUP BY department with MAX(salary) when employee details are not needed.

Q: Employees earning more than their manager.

Answer / Solution: Self-join employee e to employee m on e.manager_id = m.id and filter e.salary > m.salary.

Q: INNER JOIN vs LEFT JOIN.

Answer / Solution: INNER JOIN returns matching rows from both sides. LEFT JOIN keeps all left rows and fills unmatched right columns with NULL.

Q: WHERE vs HAVING.

Answer / Solution: WHERE filters rows before grouping; HAVING filters groups after aggregation.

Q: GROUP BY vs ORDER BY.

Answer / Solution: GROUP BY creates groups for aggregation. ORDER BY sorts the final result.

Q: Self join example.

Answer / Solution: Join a table to itself to model relationships such as employee-manager or category-parent.

Q: Find duplicate records.

Answer / Solution: GROUP BY the business key and use HAVING COUNT(*) > 1, or use ROW_NUMBER() to identify rows beyond the first.

Q: Top 3 salaries per department.

Answer / Solution: Use DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) and filter rank <= 3.

Q: Running total.

Answer / Solution: Use SUM(amount) OVER (PARTITION BY account_id ORDER BY transaction_time ROWS UNBOUNDED PRECEDING).

Q: Find employees without a department.

Answer / Solution: Use LEFT JOIN department and filter WHERE department.id IS NULL, or use NOT EXISTS.

Q: EXISTS vs IN.

Answer / Solution: Both can express membership. EXISTS can be efficient for correlated existence checks; optimizer behavior varies, so inspect the plan.

Q: What is a window function?

Answer / Solution: It calculates across related rows without collapsing them into one row per group, using OVER(PARTITION BY ... ORDER BY ...).

Q: How do you prevent SQL injection?

Answer / Solution: Use parameterized queries/prepared statements, validate input, avoid string concatenation for SQL, and apply least-privilege database accounts.

8. Coding Problems with Java Solutions

Q: Reverse a String.

Answer / Solution: Solution: use new StringBuilder(s).reverse().toString(). For an interview, also explain a two-pointer char-array approach if asked to avoid library reverse.

Q: Check palindrome.

Answer / Solution: Solution: compare characters from both ends: for (int l=0,r=s.length()-1; l<r; l++,r--) if (s.charAt(l)!=s.charAt(r)) return false; return true.

Q: Find first non-repeating character.

Answer / Solution: Solution: count with LinkedHashMap<Character,Integer>, then return the first entry whose count is 1. Time O(n), space O(k).

Q: Find duplicate characters.

Answer / Solution: Solution: use a HashSet to track seen characters; when add() returns false, the character is duplicate.

Q: Remove duplicates from an integer array.

Answer / Solution: Solution: Arrays.stream(arr).distinct().toArray(); explain that a LinkedHashSet preserves encounter order if a collection solution is preferred.

Q: Find missing number from 1..n.

Answer / Solution: Solution: expected = n*(n+1)/2; subtract the array sum. For overflow-safe code, use long.

Q: Find second-largest distinct number.

Answer / Solution: Solution: track largest and second-largest in one pass, updating only for distinct values. Handle arrays with fewer than two distinct values.

Q: Move zeros to end.

Answer / Solution: Solution: maintain write index; copy non-zero values forward, then fill remaining positions with zero. $O(n)$ time and $O(1)$ extra space.

Q: Check anagrams.

Answer / Solution: Solution: normalize as required, count characters with a map/array, and compare counts. Sorting both strings is simpler but $O(n \log n)$.

Q: Count character frequency.

Answer / Solution: Solution: use `HashMap.merge(c, 1, Integer::sum)`, or `groupingBy(Function.identity(), counting())` with streams.

Q: Reverse words in a sentence.

Answer / Solution: Solution: split on whitespace, iterate from the last word to the first, and join. Mention handling multiple spaces if required.

Q: Longest consecutive sequence.

Answer / Solution: Solution: put values in a `HashSet`; for each x where $x-1$ is absent, grow $x+1$ until missing. Expected $O(n)$.

Q: Find intersection of two arrays.

Answer / Solution: Solution: put one array in a `HashSet` and scan the other, collecting values that exist in the set.

Q: Two Sum.

Answer / Solution: Solution: maintain `Map<value,index>`; for each x , check whether `target-x` exists. $O(n)$ expected time.

Q: Merge two sorted arrays.

Answer / Solution: Solution: use two pointers and fill the result with the smaller current value; $O(n+m)$.

Q: Find maximum subarray sum.

Answer / Solution: Solution: Kadane's algorithm: `current = max(x, current+x)`, `best = max(best,current)`. $O(n)$.

Q: Frequency of words.

Answer / Solution: Solution: split normalized text and use `groupingBy(Function.identity(), counting())` or a `HashMap`.

Q: Group employees by department using streams.

Answer / Solution: Solution: `employees.stream().collect(Collectors.groupingBy(Employee::getDepartment))`.

Q: Highest salary by department using streams.

Answer / Solution: Solution: `groupingBy(Employee::getDepartment, collectingAndThen(maxBy(Comparator.comparing(Employee::getSalary)), Optional::get))`.

Q: Second-highest salary using streams.

Answer / Solution: Solution: `employees.stream().map(Employee::getSalary).distinct().sorted(Comparator.reverseOrder()).skip(1).findFirst()`.

Q: Convert list to map with duplicate IDs.

Answer / Solution: Solution: `Collectors.toMap(Employee::getId, Function.identity(), (a,b) -> b)` if the business rule is last-write-wins.

Q: Flatten nested lists.

Answer / Solution: Solution: `nested.stream().flatMap(Collection::stream).collect(Collectors.toList())`.

Q: Find employees above average salary.

Answer / Solution: Solution: calculate average first, then filter employees where salary > average. For large datasets, consider doing it in SQL.

Q: Sort employees by salary descending then name.

Answer / Solution: Solution:
`sorted(Comparator.comparing(Employee::getSalary).reversed()).thenComparing(Employee::getName))`.

Q: Thread-safe counter.

Answer / Solution: Solution: use `AtomicInteger.incrementAndGet()` or `LongAdder` for very high update contention; avoid plain `int++` across threads.

9. Production, Architecture & Senior Scenarios

Q: API suddenly becomes slow. What do you do?

Answer / Solution: Check latency/error metrics, recent deployments, traffic, thread pools, CPU/memory/GC, DB latency, external dependencies, logs and traces. Isolate the bottleneck before changing code.

Q: A service takes 10 seconds to respond.

Answer / Solution: Trace the request, identify the slow span, check timeout/retry multiplication, DB queries, downstream services, connection pools, and thread saturation.

Q: Database CPU is 90%.

Answer / Solution: Inspect active queries and execution plans, expensive joins/sorts, missing/unused indexes, lock contention, connection volume, and recent workload changes. Fix the highest-impact cause and benchmark.

Q: OutOfMemoryError investigation.

Answer / Solution: Capture heap dump, inspect retained objects/dominators, check GC logs and heap usage trend, identify leaks/unbounded caches/large payloads, and verify container/JVM limits.

Q: 1000 concurrent requests hit one API.

Answer / Solution: Make the endpoint stateless, use bounded thread/connection pools, caching where appropriate, async processing for long work, rate limiting, load balancing, and backpressure.

Q: Two services update separate databases.

Answer / Solution: Avoid distributed locking where possible. Use Saga/outbox/event-driven consistency and compensating actions; define clear business consistency requirements.

Q: Message processed twice.

Answer / Solution: Assume at-least-once delivery. Use an idempotency key/event ID with a unique constraint or inbox table and commit business state plus deduplication atomically.

Q: Design highly available REST API.

Answer / Solution: Run multiple instances behind a load balancer, externalize state, use health checks, resilient downstream calls, replicated databases, observability, automated deployment, and tested failover.

Q: Secure a microservices platform.

Answer / Solution: Use strong identity, TLS, authorization, secret management, least privilege, network segmentation, secure headers, audit logs, dependency scanning, and centralized security policies.

Q: Explain your current project architecture.

Answer / Solution: Start with business flow, then clients/API gateway, services, data stores, messaging, security, deployment and observability. Explain your personal ownership and one design trade-off.

Q: How do you handle production incidents?

Answer / Solution: Stabilize first, communicate impact, gather evidence, mitigate safely, preserve logs/metrics, identify root cause, then implement and verify corrective/preventive actions.

Q: How do you improve API performance?

Answer / Solution: Measure first; optimize DB queries/indexes, payloads, serialization, connection pools, caching, downstream calls, and concurrency. Validate with load tests.

Q: How do you design for scalability?

Answer / Solution: Keep services stateless, scale horizontally, partition workloads, use queues for spikes, optimize persistence, cache carefully, and monitor saturation.

Q: How do you perform a code review as a senior engineer?

Answer / Solution: Check correctness, security, concurrency, API compatibility, performance, tests, maintainability, observability and operational impact—not just formatting.

Q: How do you mentor junior developers?

Answer / Solution: Set clear goals, pair on design/coding, review constructively, explain trade-offs, encourage tests and documentation, and gradually increase ownership.

10. Advanced Senior Java / System Design

Q: What is SOLID?

Answer / Solution: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. Apply them pragmatically rather than creating unnecessary abstractions.

Q: Dependency Injection vs Service Locator.

Answer / Solution: Dependency Injection supplies dependencies from outside and makes dependencies explicit. Service Locator hides dependencies behind a registry and is generally harder to test and reason about.

Q: Factory vs Strategy pattern.

Answer / Solution: Factory chooses/creates an implementation. Strategy encapsulates interchangeable algorithms and lets callers select behavior.

Q: Builder pattern use case.

Answer / Solution: Useful for constructing complex immutable objects with many optional parameters while keeping constructors readable.

Q: Template Method vs Strategy.

Answer / Solution: Template Method fixes an algorithm skeleton through inheritance; Strategy varies an algorithm through composition.

Q: What is defensive copying?

Answer / Solution: Copy mutable input when storing it and copy mutable state when returning it, preventing callers from modifying internal state.

Q: What is memory leak in Java?

Answer / Solution: Objects are no longer logically needed but remain strongly reachable, preventing garbage collection. Common causes include static collections, listeners, caches, and ThreadLocal misuse.

Q: Heap vs stack.

Answer / Solution: Heap stores objects and is managed by GC. Each thread has a stack containing call frames, local variables and references used during method execution.

Q: Young vs old generation.

Answer / Solution: Most newly allocated objects start in young generation; surviving objects may be promoted to old generation. GC behavior depends on the selected collector and JVM version.

Q: What is garbage collection?

Answer / Solution: The JVM reclaims unreachable objects. Modern collectors balance pause times, throughput, and memory footprint.

Q: What is class loading?

Answer / Solution: Class loaders locate and load class bytecode, while linking and initialization prepare it for use. Java uses a delegation model and multiple loader layers.

Q: What is reflection?

Answer / Solution: Reflection allows runtime inspection and invocation of classes, methods and fields. It is powerful but can reduce type safety and affect performance/maintainability.

Q: What is an annotation?

Answer / Solution: Metadata attached to code elements. Frameworks can inspect annotations at compile time or runtime to apply behavior.

Q: What is a sealed class?

Answer / Solution: A sealed class restricts which classes may directly extend or implement it, enabling stronger domain modeling and exhaustive reasoning.

Q: What is the Java memory model?

Answer / Solution: It defines visibility, ordering and atomicity rules for shared-memory concurrency, including happens-before relationships.

Q: What is happens-before?

Answer / Solution: A guarantee that one action's effects are visible to another action in the required ordering, established by mechanisms such as synchronization, volatile writes/reads, and thread start/join.

Q: What is connection pool exhaustion?

Answer / Solution: All DB connections are occupied or leaked, so new requests wait or fail. Check pool metrics, query latency, transaction scope, leaks and pool sizing.

Q: What is backpressure?

Answer / Solution: A mechanism that prevents producers from overwhelming consumers by limiting or slowing incoming work.

Q: What is contract testing?

Answer / Solution: Testing service/API compatibility against an agreed contract so producers and consumers can evolve safely.

Q: What is blue-green deployment?

Answer / Solution: Maintain two environments; deploy to the idle environment, validate it, then switch traffic. Rollback is usually a traffic switch.

Q: What is canary deployment?

Answer / Solution: Release to a small percentage of traffic first, observe health/metrics, and gradually increase exposure.

Q: How would you design a high-volume payment API?

Answer / Solution: Use idempotency, strong authentication/authorization, transactional state changes, unique business references, durable messaging where needed, strict audit trails, timeouts, reconciliation, encryption, observability, and carefully defined consistency guarantees.

11. HCL TP1 Mock Interview — Ideal Answers

Mock Q1: Tell me about yourself.

Ideal answer: I am a Java full-stack engineer with strong experience in Java, Spring Boot, REST APIs, Angular, SQL and production support. My strength is building maintainable backend services, troubleshooting production issues, and working across design, development, testing and deployment. In recent projects I have worked on financial/payment-related applications where security, reliability and auditability are important. I also contribute to code reviews, technical design and mentoring.

Mock Q2: Explain your current project architecture.

Ideal answer: I would explain it from the business flow rather than listing technologies. The client calls the API layer, authentication and authorization are applied, the request reaches the appropriate service, business rules are executed, persistence is handled through the repository/data layer, and asynchronous work is sent through messaging where appropriate. I would then explain deployment, CI/CD, monitoring and my specific ownership. Finally, I would mention one important design decision and its trade-off.

Mock Q3: Why did you choose Spring Boot?

Ideal answer: It reduces configuration through starters and auto-configuration, provides embedded server support, integrates well with Spring's dependency injection and transaction management, and has strong production tooling through Actuator and the wider Spring ecosystem. It lets the team focus more on business functionality while retaining control through explicit configuration when required.

Mock Q4: Explain HashMap internal working.

Ideal answer: HashMap uses the key's hash to determine a bucket. It compares keys using hash information and equals(). Collisions are stored in the same bucket, and modern Java implementations can convert heavily populated buckets into tree structures under appropriate conditions. Correct equals/hashCode implementation and immutable key state are critical.

Mock Q5: How would you troubleshoot a slow REST API?

Ideal answer: First I establish whether the slowdown is application-wide or endpoint-specific. I check latency, error rate, throughput, CPU, memory/GC, thread and connection pools, logs and distributed traces. Then I isolate the slow layer—database, downstream service, serialization, lock contention or application code. I fix the measured bottleneck and validate the change with load/performance tests.

Mock Q6: Explain @Transactional and a common pitfall.

Ideal answer: It creates a transaction boundary, normally through a Spring proxy. A common pitfall is self-invocation: calling a transactional method from another method in the same class can bypass the proxy, so the transaction advice may not execute as expected. Another pitfall is using overly large transaction scopes that hold DB resources too long.

Mock Q7: How do you design a resilient microservice?

Ideal answer: Use bounded timeouts, appropriate retries with exponential backoff, circuit breakers, bulkheads where useful, idempotency, graceful degradation, health checks, observability and clear failure contracts. Avoid retry storms and make sure the service does not block valuable threads on slow dependencies.

Mock Q8: Explain Saga with an example.

Ideal answer: Suppose an order service creates an order, payment service authorizes payment, and inventory reserves stock. Each service commits its own local transaction. If inventory fails after payment succeeds, a compensating action can cancel/refund the payment. Orchestration centralizes the workflow; choreography uses events. The choice depends on complexity and governance.

Mock Q9: How do you prevent duplicate payment/message processing?

Ideal answer: I assume at-least-once delivery. I use a business idempotency key or event ID, persist it with a unique constraint/inbox record, and make the business update and deduplication decision atomic. Retries then become safe. For external side effects, I also use reconciliation and durable audit records.

Mock Q10: What is your approach to production incidents?

Ideal answer: Stabilize first: assess impact and stop further damage. Communicate clearly, collect metrics/logs/traces and reproduce if safe. Apply the smallest safe mitigation, verify recovery, then perform root-cause analysis. Finally, add a corrective fix, monitoring/alerting and a regression test or runbook update.

Mock Q11: What makes you a senior engineer?

Ideal answer: I focus not only on implementation but also on reliability, maintainability, security, performance and business impact. I make trade-offs explicit, communicate risks early, help the team converge on designs, review code constructively, mentor others and take ownership through production.

Mock Q12: How would you improve a database-heavy API?

Ideal answer: Measure query latency and execution plans first. Remove N+1 queries, fetch only required columns, add or adjust indexes based on actual access patterns, paginate large results, tune connection pools and transaction boundaries, and consider caching where data freshness permits. I validate improvements with realistic load tests.

Mock Q13: Explain optimistic locking.

Ideal answer: The entity has a version value. A read gets version N; an update succeeds only if the database row still has version N, then increments it. If another transaction changed the row, the update affects zero rows and the application detects a conflict. It works well when conflicts are relatively uncommon.

Mock Q14: How would you secure a REST API?

Ideal answer: Use TLS, strong authentication such as OAuth2/OIDC where appropriate, authorization based on scopes/roles, input validation, secure secret storage, rate limiting, audit logging, safe error responses and dependency patching. JWT validation must check signature, issuer, audience and expiry rather than merely decoding the token.

Mock Q15: Coding: find second-highest salary.

Ideal answer: For distinct salary:
`employees.stream().map(Employee::getSalary).distinct().sorted(Comparator.reverseOrder()).skip(1).findFirst()`. This is concise, but in a production system I would often push aggregation to the database if the dataset is large.

Mock Q16: Coding: longest consecutive sequence.

Ideal answer: Put all numbers into a HashSet. For each x, start only when x-1 is absent, then count x, x+1, x+2... until the sequence ends. This gives expected O(n) time and O(n) space.

Mock Q17: Coding: first non-repeating character.

Ideal answer: Count characters in insertion order using LinkedHashMap, then scan entries and return the first with count 1. Complexity is O(n) time and O(k) space.

Mock Q18: Why should we hire you?

Ideal answer: I bring strong Java/Spring engineering fundamentals together with practical production ownership. I can contribute to coding immediately, but I can also reason about architecture, failure modes, performance, security and operational concerns. I communicate clearly and can work effectively across backend, frontend, database and delivery teams.

Mock Q19: Do you have questions for us?

Ideal answer: Yes. I would ask: What are the most important technical challenges for this role in the first six months? How is architecture ownership distributed? What are the main reliability/performance goals? How are technical decisions and code quality measured? What would success look like for the person joining this role?

12. Final 7-Day Revision Plan

Day 1 — Core Java, OOP, String, equals/hashCode, Collections and HashMap internals.

Day 2 — Java 8+, Streams, Optional, functional interfaces and 10 coding problems.

Day 3 — Multithreading, ExecutorService, CompletableFuture, locks, volatile and thread dumps.

Day 4 — Spring Boot, DI, bean lifecycle, transactions, validation, exception handling and Actuator.

Day 5 — REST, microservices, Gateway, discovery, Feign, Circuit Breaker, Saga, JWT and resilience.

Day 6 — SQL, JPA/Hibernate, N+1, locking, indexes, performance and database migration.

Day 7 — Mock interview: explain project architecture, solve 5 coding problems, practice 10 production scenarios and prepare concise project stories.

Interview answer formula: Definition → Internal working → Project example → Trade-off → Failure/edge case → Testing/monitoring.